# day08_磁盘管理

今日内容：

- 1- 【作业】日志轮替的实战二
- 2- 【了解】磁盘管理的基本介绍
- 3- 【了解】磁盘结构介绍
- 4- 【了解】常见的文件系统， 不同文件系统的差异
- 5- 【记录】常用磁盘管理命令
- 6- 【实操】如何进行磁盘的分区操作
- 7- 【实操】LVM逻辑卷管理
- 8- 【理解】RAID方案

## 1. 磁盘管理的基本介绍

磁盘管理是操作系统中用于配置和维护存储设备（如硬盘、SSD、RAID 等）的一套工具和方法。它包括对存储设备的分区、格式化、挂载、卷管理等操作，以确保数据可以被操作系统和用户正常访问和存储。

在 CentOS Stream 9 中，磁盘管理主要包括以下方面：

- 磁盘分区:

```
磁盘分区是将磁盘划分为多个逻辑区域，方便不同用途的数据存储。分区类型包括：

MBR（Master Boot Record）：传统分区表，最多支持 4 个主分区，单个磁盘最大支持 2TB。
GPT（GUID Partition Table）：现代分区表，支持更多分区和更大的磁盘容量。

常用工具：
fdisk：用于管理 MBR 分区表的工具。
gdisk：用于管理 GPT 分区表的工具。
```

- 文件系统管理:

```
文件系统是存储数据的逻辑结构，常见文件系统包括：

```

```
Ext4：Linux 系统默认文件系统，稳定性和性能较好。
XFS：CentOS 默认文件系统，适用于大规模存储。
Btrfs：支持快照和压缩功能的新型文件系统。但是稳定性相对于XFS和Ext4较差，不建议在生产环境使用


相关命令：
mkfs：创建文件系统（格式化操作）（如 mkfs.ext4）。
blkid 和 lsblk：查看磁盘和分区的文件系统类型
```

- 挂载和卸载【已学习】

```
挂载（Mount）是将磁盘或分区连接到操作系统的目录结构中，使其可用。

相关命令：
mount：手动挂载分区到目录。
umount：卸载已挂载的分区。
/etc/fstab：定义自动挂载的配置文件。
```

- 逻辑卷管理（LVM）

```
逻辑卷管理是一种灵活的磁盘管理方式，适合动态调整磁盘容量的场景。

LVM 的基本概念：

PV（Physical Volume）：物理卷，磁盘或分区的抽象。
VG（Volume Group）：卷组，由一个或多个 PV 组成。
LV（Logical Volume）：逻辑卷，可以创建文件系统并挂载。

相关命令：
pvcreate、vgcreate、lvcreate：分别创建 PV、VG 和 LV。
lvextend 和 lvreduce：扩展或缩小逻辑卷。
```

- RAID（Redundant Array of Independent Disks，独立磁盘冗余阵列）管理

```
RAID 是一种将多个物理硬盘组合成一个逻辑单元的技术，通过并行操作和冗余备份来提高数据的性能、可用性和容错能力。

包括有RAID0、RAID1、RAID5、RAID6、RAID10等
```

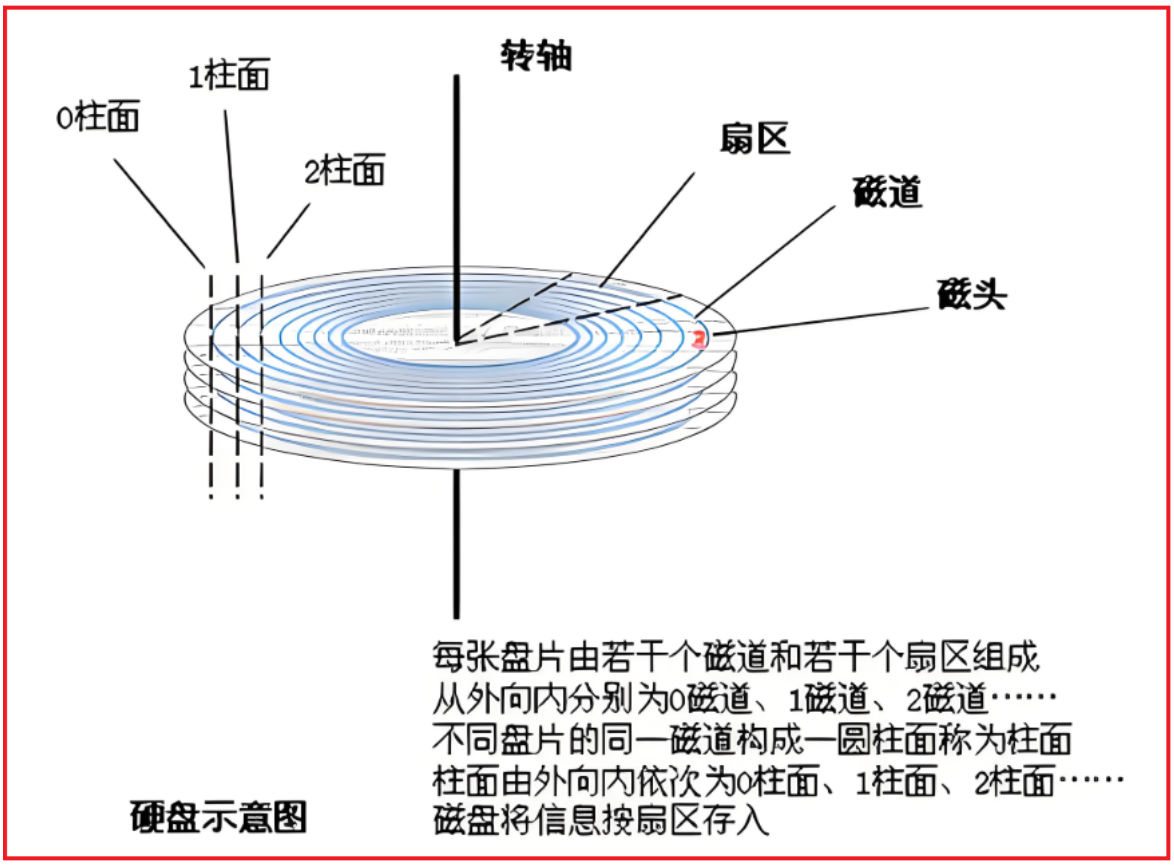
# 2. 磁盘结构介绍

## 2.1 硬盘的逻辑结构

磁盘物理结构：




磁盘逻辑结构：



默认情况下，每个扇区的大小是固定的，为 512Byte。扇区也是磁盘的最小存贮单位。

硬盘的大小是使用“磁头数×柱面数×扇区数×每个扇区的大小”这样的公式来计算的。其中磁头数（Heads）表示硬盘总共有几个磁盘，也可以理解成为硬盘有几个盘面，然后乘以二；柱面数（Cylinders）表示硬盘每一面盘片有几条磁道；扇区数（Sectors）表示每条磁道上有几个扇区；每个扇区的大小一般是 512Byte。

## 2.2 磁盘接口

磁盘的接口决定了它与主机（如计算机、服务器）之间的数据传输方式和性能

- SATA（Serial ATA）接口

```
特点：  目前最常见的接口类型，适用于大部分消费级HDD(机械磁盘)和SSD(固态磁盘)。
传输速率：
    SATA I：1.5 Gbps（理论最大 150 MB/s）
    SATA II：3.0 Gbps（理论最大 300 MB/s）
    SATA III：6.0 Gbps（理论最大 600 MB/s）
兼容性强，支持热插拔。

用途：
    普通机械硬盘（HDD）
    一些基于SATA协议的SSD（SATA SSD）
```




- SAS（Serial Attached SCSI）接口

```
特点：
    主要用于企业级存储设备，性能和可靠性优于SATA。
    支持双端口、热插拔，适合高负载环境。
传输速率：
    SAS 1.0：3.0 Gbps。
```

```
    SAS 2.0: 6.0 Gbps。
    SAS 3.0: 12.0 Gbps。
用途:
        数据中心和服务器的机械硬盘和企业级SSD
```

SATA




SATA接口是断开的

SAS




SAS接口是连接的

- 3 NVMe接口

```
特点：专为SSD设计，利用PCIe总线直接连接CPU。传输速率比SATA快很多：
PCIe 3.0 x4：理论最大 32 Gbps（约 4 GB/s）。
PCIe 4.0 x4：理论最大 64 Gbps（约 8 GB/s）。
PCIe 5.0 x4：理论最大 128 Gbps（约 16 GB/s）。
更低的延迟和更高的并发性能。

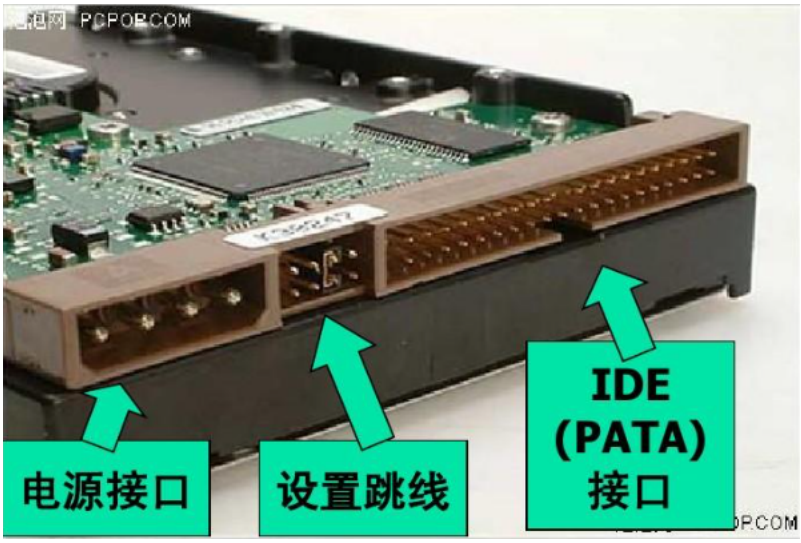
用途：
        高性能固态硬盘（NVMe SSD），广泛用于高端PC、服务器和数据中心。
```




- 4- IDE（Integrated Drive Electronics）接口（过时）

```
特点：
    一种老式的并行接口，也称为PATA（Parallel ATA）。
    数据速率较低，最大为 133 MB/s。
    已被SATA完全取代。
```

```
用途：
        早期的机械硬盘和光驱。
```




## 3. 文件系统

文件系统的主要作用是将存储设备（如硬盘、SSD）上的物理空间组织为逻辑单元，提供数据存储、检索和管理的标准方法。一般文件系统主要由超级块（Superblock）、inode、数据块、目录结构构成的

```
超级块（Superblock）：记录文件系统的元信息（如大小、类型）
inode：存储文件的元数据（如大小、权限）
数据块：实际存储文件内容的地方
目录结构：用于组织和索引文件
```

常见文件系统的对比：

| 文件系统 | 特点 | 最大单文件大小 | 最大卷大小 | 优势 | 劣势 | 适用场景 |
| --- | --- | --- | --- | --- | --- | --- |
| **Ext4** | 默认文件系统，稳定性和兼容性好，支持延迟分配（delayed allocation）和日志功能。 | 16TB | 1EB | - 稳定性强 - 性能平衡 - 支持大文件和卷 - 广泛兼容性 | - 缺乏高级功能（如原生快照） | 通用 Linux 系统；桌面和服务器。 |
| **XFS** | 高性能 64 位日志文件系统，支持并行 I/O 操作和动态分配 inode。 | 8EB | 8EB | - 性能优越，适合大规模存储 - 文件系统扩展方便 - 默 | - 不支持文件系统收缩 - 数据修复工具较少 | 数据库、虚拟化、NAS 存储。 |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | 认用于 CentOS Stream 9 | | |
| **Btrfs** | 新型文件系统，支持快照、子卷、压缩、克隆等高级功能。 | 16EB | 16EB | - 快照和压缩功能强大 - 数据完整性强 - 支持多设备和 RAID | - 稳定性相对较低 - 部分功能仍在开发阶段 | 实验性环境；需要快照的场景。 |
| **F2FS** | 针对闪存存储（如 SSD 和 eMMC）优化，减少磨损，延长寿命。 | 4TB | 16TB | - 优化闪存存储 - 性能出色（随机读写快） | - 社区支持不如 Ext4 和 XFS - 稳定性和功能较简单 | SSD 和嵌入式存储设备。 |
| **ZFS** | 先进的功能集合，包括快照、压缩、复制、数据完整性校验和内置 RAID。 | 16EB | 16EB | - 快照和压缩强大 - 数据完整性保障 - 支持在线扩展和分区 | - 高内存消耗 - 开源社区整合性较低，需额外安装（未内置） | 数据库、存档存储、云存储。 |
| **VFAT** | 支持多平台（Windows 和 Linux），通常用于可移动存储设备。 | 4GB | 2TB | - 跨平台兼容性强 | - 不支持权限管理 - 不适合大文件或复杂结构 | USB 闪存、外置硬盘。 |
| **NTFS** | Windows 默认文件系统，支持大文件、大容量存储，并且具有权限管理功能。 | 16EB | 16EB | - 跨平台兼容性（需安装驱动） - 支持文件压缩和加密 | - 在 Linux 中性能较低（需 `ntfs-3g` 驱动） | 与 Windows 共享数据。 |
| **ISO9660** | 只读文件系统，主要用于光盘存储（如 CD/DVD）。 | 4GB | 2TB | - 标准化的光盘文件系统 - 易于创建和读取 | - 只读特性 - 不支持大文件 | 光盘或镜像存储。 |

```
重点说明：
    Ext4 和 XFS 是 CentOS 中最常用的文件系统。Ext4 是通用选择，XFS 在高性能和大规模存储中更优。
    Btrfs 和 ZFS 提供丰富的高级功能，但相对复杂，适用于特定应用场景。
    F2FS 对于闪存设备（如 SSD）提供了优化。
    NTFS 和 VFAT 是跨平台场景下的选择，但 Linux 中性能或功能有限。
实践建议：
    生产环境：选择 Ext4或 XFS（大文件/高性能）。
    高级需求：尝试 Btrfs 或 ZFS。
```

```
9       跨平台：使用 NTFS 或 VFAT。
```

## 4. 常用的磁盘管理命令

- 1- df命令

```
1   df [选项]
2
3   # -a 显示特殊文件系统，这些文件系统几乎都是保存在内存中的。如/proc，因为是挂载在内存中，
    所以占用量都是 0
4   # -h 单位不再只用 KB，而是换算成习惯单位
5   # -T 多出了文件系统类型一列
6
7   常用组合： df -h
```

```
[root@node3 ~]# df -ahT
文件系统                类型        容量  已用  可用 已用% 挂载点
proc                    proc           0     0     0     - /proc
sysfs                   sysfs          0     0     0     - /sys
devtmpfs                devtmpfs    4.0M     0  4.0M    0% /dev
securityfs              securityfs     0     0     0     - /sys/kernel/security
tmpfs                   tmpfs       1.8G     0  1.8G    0% /dev/shm
devpts                  devpts         0     0     0     - /dev/pts
tmpfs                   tmpfs       726M  9.1M  717M    2% /run
cgroup2                 cgroup2        0     0     0     - /sys/fs/cgroup
pstore                  pstore         0     0     0     - /sys/fs/pstore
bpf                     bpf            0     0     0     - /sys/fs/bpf
/dev/mapper/cs_bogon-root xfs         95G  3.8G   92G    4% /
systemd-1               autofs         0     0     0     - /proc/sys/fs/binfmt_misc
mqueue                  mqueue         0     0     0     - /dev/mqueue
hugetlbfs               hugetlbfs      0     0     0     - /dev/hugepages
debugfs                 debugfs        0     0     0     - /sys/kernel/debug
tracefs                 tracefs        0     0     0     - /sys/kernel/tracing
fusectl                 fusectl        0     0     0     - /sys/fs/fuse/connections
configfs                configfs       0     0     0     - /sys/kernel/config
none                    ramfs          0     0     0     - /run/credentials/systemd-sysctl.service
none                    ramfs          0     0     0     - /run/credentials/systemd-tmpfiles-setup-dev.service
/dev/nvme0n1p1          xfs          960M  233M  728M   25% /boot
none                    ramfs          0     0     0     - /run/credentials/systemd-tmpfiles-setup.service
sunrpc                  rpc_pipefs     0     0     0     - /var/lib/nfs/rpc_pipefs
tmpfs                   tmpfs       363M  4.0K  363M    1% /run/user/0
```

- 2- du命令

```
1   du [选项] [目录或文件名]
2
3   选项：
4       -a 显示每个子文件的磁盘占用量。默认只统计子目录的磁盘占用量
5       -h 使用习惯单位显示磁盘占用量，如 KB，MB或 GB等
6       -s 统计总占用量，而不列出子目录和子文件的占用量
7   常用组合： du -sh 目录或文件名
8
9   du与 df的区别：
10      du是用于统计文件大小的，统计的文件大小是准确的；
```

```
11      df是用于统计空间大小的，统计的剩余空间是准确的
```

```
[root@node3 ~]# du -sh /root/
80K     /root/
```

- 3- 列出块设备的树状结构，包括磁盘、分区、挂载点：

```
1   lsblk
```

```
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
```

- 4- 查看分区的 UUID 和文件系统类型

```
1   blkid [设备名称]
```

```
[root@node3 ~]# blkid
/dev/mapper/cs_bogon-root: UUID="9c026495-f978-4b58-8c91-1881219ba215" TYPE="xfs"
/dev/mapper/cs_bogon-swap: UUID="8c74558c-490f-46d7-a86d-de9279627bc1" TYPE="swap"
/dev/nvme0n1p1: UUID="f683bec8-a33e-4ccf-9fcf-b9a8a876cf39" TYPE="xfs" PARTUUID="7a267dd2-01"
/dev/nvme0n1p2: UUID="dLZM86-E2A0-qwdG-Ri7Y-Hxjh-YhS2-Wrmm7k" TYPE="LVM2_member" PARTUUID="7a267dd2-02"
/dev/sr0: UUID="2024-09-23-03-56-22-00" LABEL="CentOS-Stream-9-BaseOS-x86_64" TYPE="iso9660" PTUUID="05aa502a" PTTYPE="dos"
[root@node3 ~]#
```

## 5. 磁盘分区操作

Linux系统中有两种常见的分区表 MBR分区表（主引导记录分区表）和 GPT分区表（GUID分区表）

| 属性 | MBR | GPT |
| --- | --- | --- |
| **引入时间** | 1983 年，引入于 IBM PC DOS 2.0。 | 2000 年，由 Intel 引入，用于替代 MBR。 |
| **最大支持容量** | 单磁盘最大 2TB。 | 理论上支持 8ZB（Zettabyte），实践中远大于 2TB。 |
| **最大分区数** | 4 个主分区，或 3 个主分区 + 1 个扩展分区（扩展分区内可以有逻辑分区）。 | 无限制（常见实现中最多支持 128 个分区）。 |
| | | |

| | | |
|---|---|---|
| **启动支持** | 仅支持 BIOS 系统引导。 | 支持 BIOS 和 UEFI 系统引导。 |
| **数据安全性** | 分区表无冗余备份，分区表损坏可能导致数据丢失。 | 分区表有多个备份，支持 CRC 校验，数据更安全。 |
| **兼容性** | 兼容性高，适用于旧操作系统。 | 与现代操作系统兼容，适合新设备。 |

其中关于MBR的分区方式，我们在前序的课程中已经实践过了，这里仅简单回顾

```
使用 fdisk 管理 MBR 分区

fdisk 设备名称
输入后进入交互式界面：

按 n 创建新分区。
选择 主分区（primary）。
按提示输入分区号和起始/结束扇区（默认即可）。
按 w 保存并退出。
查看分区
lsblk 设备名称
```

---

## 5.1 GPT分区实操

GPT分区实操[node3实操]：

前置操作： 首先在node3关机的状态下， 挂载一个新磁盘， 磁盘大小为20GB

- 1- 确认磁盘信息：

```
使用以下命令查看系统中可用磁盘：
lsblk
```

```
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
nvme0n2             259:3    0   20G  0 disk
```

- 2- 进入GPT交互窗口：parted 设备名称

```
格式:
    parted 设备名称
示例:
parted /dev/nvme0n2
```

```
[root@node3 ~]# parted /dev/nvme0n2
GNU Parted 3.5
使用 /dev/nvme0n2
欢迎使用 GNU Parted! 输入 'help' 来查看命令列表。
(parted)
```

进入交互式命令后：

- 2.1 创建GPT分区表

```
mklabel gpt

注意:
        mklabel后面指定的分区表的类型，在Linux主要使用gpt或者msdos,gpt表示采用GPT方式来设置分区
        如果设置分区表类型为  msdos  表示依然使用MBR的方式来进行分区方案
```

- 2.2 创建一个新分区

```
# 创建一个分区
mkpart primary xfs 1MiB 100%

说明:
    mkpart: parted 的命令，用于创建一个新分区。
```

```
    primary: 指定分区类型为主分区（适用于 GPT，GPT 中主分区和扩展分区的概念已弱化）。
    xfs: 指定分区使用的文件系统类型（仅作为标记，实际格式化需另行操作）。可以省略
    1MiB: 指定分区起始位置，单位为 MiB（兆字节）。通常会从 1MiB 开始分区，这是为了确保分区的起始位置不与磁盘的 保护性MBR（主引导记录）和 对齐要求 相冲突
    100%: 指定分区结束位置，占用整个磁盘剩余空间（100% 表示从起始位置到磁盘尾部）。

如何设置多个呢？例如 5个分区
mkpart primary xfs 1MiB 20%
mkpart primary xfs 20% 40%
mkpart primary xfs 40% 60%
mkpart primary xfs 60% 80%
mkpart primary xfs 80% 100%

或者也可以采用 GB的方式
mkpart primary xfs 1MiB 4GiB
mkpart primary xfs 4GiB 8GiB
mkpart primary xfs 8GiB 12GiB
mkpart primary xfs 12GiB 16GiB
mkpart primary xfs 16GiB 100%
```

- 2.3 打印分区信息

```
print
```

```
(parted) print
型号：VMware Virtual NVMe Disk (nvme)
磁盘 /dev/nvme0n2：21.5GB
扇区大小 (逻辑/物理)：512B/512B
分区表：gpt
磁盘标志：

编号  起始点  结束点  大小  文件系统  名称  标志

(parted) mkpart primary xfs 1MiB 20%
(parted) mkpart primary xfs 20% 40%
(parted) mkpart primary xfs 40% 60%
(parted) mkpart primary xfs 60% 80%
(parted) mkpart primary xfs 80% 100%
(parted) print
型号：VMware Virtual NVMe Disk (nvme)
磁盘 /dev/nvme0n2：21.5GB
扇区大小 (逻辑/物理)：512B/512B
分区表：gpt
磁盘标志：

编号  起始点   结束点   大小    文件系统  名称     标志
 1    1049kB   4295MB   4294MB  xfs       primary
 2    4295MB   8590MB   4295MB  xfs       primary
 3    8590MB   12.9GB   4295MB  xfs       primary
 4    12.9GB   17.2GB   4295MB  xfs       primary
 5    17.2GB   21.5GB   4294MB  xfs       primary
```

- 2.4 退出 parted

```
quit
```

- 3- 打印分区信息：

```
lsblk
```

```
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
nvme0n2             259:3    0   20G  0 disk
├─nvme0n2p1         259:9    0    4G  0 part
├─nvme0n2p2         259:10   0    4G  0 part
├─nvme0n2p3         259:11   0    4G  0 part
├─nvme0n2p4         259:12   0    4G  0 part
└─nvme0n2p5         259:13   0    4G  0 part
```

后续的格式化、挂载操作均和之前讲解格式化和挂载是一致的

注意：

在实际工作中，推荐优先选择 GPT，除非存在兼容性问题或特殊需求。

parted中所有的操作都是立即生效，没有保存生效的概念。这点和 fdisk交互命令明显不同

---

基于gdisk分区方式【了解 可以不演示】：

- 1- 第一步： 下载gdisk工具

```
dnf -y install gdisk
```

- 2- 基于gdisk完成GPT的分区

```
gdisk  设备路径

执行后，  就进入gdisk交互窗口了

进入后，常用操作：
    p：查看分区信息
    n：  增加新的分区
        分别设置分区的 开始 和 结束 范围 【与fdisk类似的】
    w：  保存退出
    q：  不保存退出
    d：  删除分区
```

## 5.2 设置交换内存

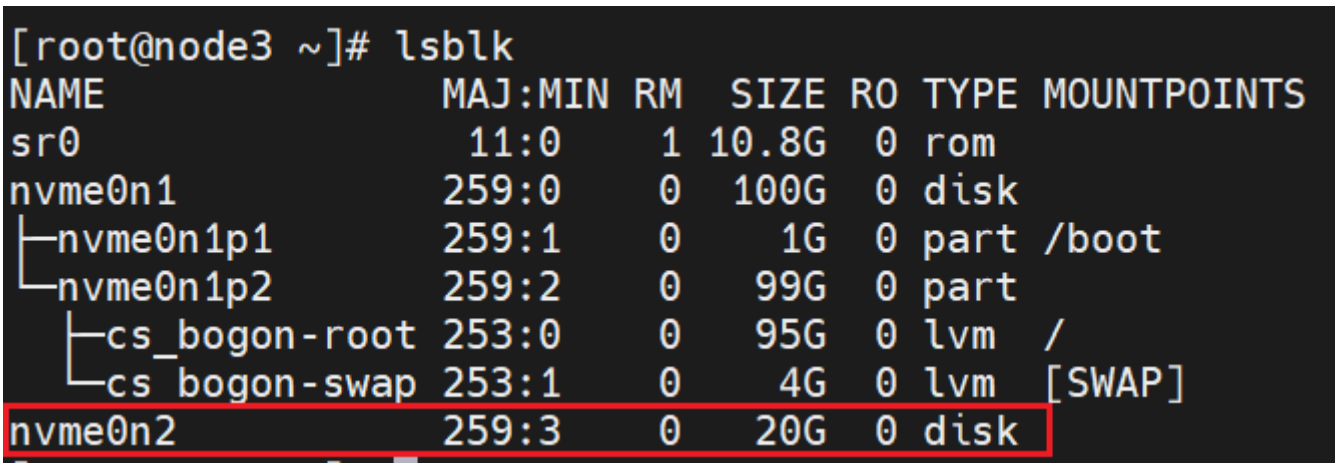
如何设置交换内存：

在 Linux 系统中，**swap** 分区用于扩展系统的虚拟内存，它是物理内存不足时用来存储临时数据的一种区域。设置 swap 分区可以帮助系统在内存压力较大的情况下提高稳定性。

- 1- 确认磁盘信息：

```
使用以下命令查看系统中可用磁盘：
lsblk


如果该磁盘已经被GPT或者MBR分区全部占用了， 请先释放出一部分的磁盘空间
```




- 2- 进入GPT交互窗口：parted 设备名称

```
parted /dev/nvme0n2
```




进入交互式命令后：

- 2.1 创建GPT分区表

```
mklabel gpt


注意：
        mklabel后面指定的分区表的类型，在Linux主要使用gpt或者msdos,gpt表示采用GPT方式来设置分区
        如果设置分区表类型为  msdos  表示依然使用MBR的方式来进行分区方案
```

- 2.2 创建swap分区： 假设你想创建一个大小为 4GB 的 swap 分区

```
 mkpart primary linux-swap 1MiB 4GiB

 # 注意：此时设置linux-swap 文件系统类型， 仅仅是一个标记， 最终还得要格式化来处理
```

- 2.3 打印分区信息：

```
(parted) print
型号：VMware Virtual NVMe Disk (nvme)
磁盘 /dev/nvme0n2：21.5GB
扇区大小 (逻辑/物理)：512B/512B
分区表：gpt
磁盘标志：

编号  起始点  结束点  大小    文件系统        名称     标志
 1    1049kB  4295MB  4294MB  linux-swap(v1)  primary  交换
```

- 2.4 退出parted

```
quit

退出后， 查看分区信息
```

```
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
nvme0n2             259:3    0   20G  0 disk
└─nvme0n2p1         259:5    0    4G  0 part
```

- 3- 格式化分区为swap

```
使用 mkswap 命令格式化分区为 swap 文件系统
mkswap /dev/nvme0n2p1


注意：只有格式化操作才是真正的将磁盘对应的分区变更为指定的文件系统
```

```
[root@node3 ~]# mkswap /dev/nvme0n2p1
正在设置交换空间版本 1，大小 = 4 GiB (4293914624 个字节)
无标签，UUID=7e2ae706-2dc1-4716-a5bf-d78c8fe92577
```

- 4- 启用swap分区

```
swapon /dev/nvme0n2p1


思考：后续如果想要关闭该交换空间?
swapoff /dev/nvme0n2p1
关闭后， 即可将该分区删除了， 后续就可以创建新的分区， 设置为普通磁盘空间
```

```
[root@node3 ~]# swapon /dev/nvme0n2p1
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
nvme0n2             259:3    0   20G  0 disk
└─nvme0n2p1         259:5    0    4G  0 part [SWAP]
[root@node3 ~]#
```

- 5- 可选： 添加到 /etc/fstab 文件中（永久保存）

```
vi /etc/fstab

添加以下内容：
/dev/nvme0n2p1 none swap sw 0 0

参数说明：
/dev/nvme0n2p1：指向你创建的 swap 分区。
none：挂载点，swap 分区不需要挂载到文件系统中，所以此处为 none。
swap：指定该分区是 swap 类型。
sw：挂载选项，表示启用 swap 分区。
0：备份选项。
0：文件系统检查顺序。

最后，记得mount -a 检查下
```

此时通过firee命令查看， 发现交换空间变大了

```
[root@node3 ~]# free
               total        used        free      shared  buff/cache   available
Mem:         3713008      556860     3070412        9284      317484     3156148
Swap:        8387576           0     8387576
[root@node3 ~]#
```

总结说明:

```
对于磁盘使用这件事来说，核心部分：添加磁盘 --> 磁盘分区 --> 磁盘分区格式化  --> 磁盘挂载使用

添加磁盘：对于物理机来说，插入一块新硬盘

```

磁盘分区：将磁盘划为一个个逻辑区域，用于存储，这部分理解为在windows中对于一块磁盘可以分为C盘 D盘 E盘

    两种方式：MBR GPT  都可以实现对我们磁盘进行分区操作，只不过分区数量\支持文件大小有所不同而已

    推荐使用分区方案：GPT

    注意：不管未来这个磁盘是作为普通存储盘 还是作为 交换空间，此部分基本没有任何区别

磁盘分区格式化：  决定这块分区最终使用什么文件系统来进行后续的数据存储工作

    需要ext4文件系统：mkfs.ext4 分区设备名称

    需要xfs文件系统：mkfs.xfs 分区设备名称

    需要交换空间系统：mkswap 分区设置名称

磁盘挂载使用：

    如果不是交换空间，统一使用mount和umount 对磁盘完成挂载和取消挂载操作

    如果该磁盘是一个交换空间，统一使用 swapon 和 swapoff 完成对磁盘的挂载(开启)和取消挂载(关闭)操作

    注意： mount和umount  或  swapon 和 swapoff  都是临时的挂载 一旦重启就没了

    如果想要永久的挂载， 就需要编辑：/etc/fstab 文件

ext4或xfs：

```
/dev/mapper/cs-root     /                       xfs     defaults        0 0
```

格式说明：

这表示系统启动时会自动挂载 `/dev/mapper/cs-root` 到 `/`，使用 `xfs` 文件系统。

/dev/mapper/cs-root：设备名称。

/：挂载点目录。

xfs：文件系统类型。

defaults：挂载选项（默认设置：读写模式、同步、允许执行文件等）。

0 0：

第一位 0：是否需要转储备份，通常设置为 0。

第二位 0：是否检查文件系统，0 表示不检查，1 表示优先检查根文件系统，2 表示检查其他文件系统。

交换空间：

```
/dev/mapper/cs-swap     none                    swap    defaults        0 0
```

参数说明：

/dev/mapper/cs-swap：指向你创建的 swap 分区。

none：挂载点，swap 分区不需要挂载到文件系统中，所以此处为 none。

swap：指定该分区是 swap 类型。

sw：挂载选项，表示启用 swap 分区。 或者可以 defaults

0：备份选项。

0：文件系统检查顺序。

```
#
# /etc/fstab
# Created by anaconda on Wed Mar 12 10:45:30 2025
#
# Accessible filesystems, by reference, are maintained under '/dev/disk/'.
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info.
#
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#                                                    针对的xfs 或者 ext4 设置模版
/dev/mapper/cs-root     /                       xfs     defaults        0 0
UUID=64536a1f-4dc7-43a3-848e-a92eb6b142e5 /boot                   xfs     defaults        0 0
/dev/mapper/cs-swap     none                    swap    defaults        0 0   针对交换空间设置模版
~
```

# 6. LVM逻辑卷管理

## 6.1 LVM的基本介绍

LVM是 Logical Volume Manager的简称，中文就是逻辑卷管理，是一种强大的磁盘管理工具。

它提供了更高的灵活性来管理磁盘空间。通过 LVM，可以对磁盘进行更灵活的分区、扩展、缩小等操作，而不需要对文件系统进行复杂的操作。LVM 是基于物理卷（Physical Volume，PV）、卷组（Volume Group，VG）和逻辑卷（Logical Volume，LV）的结构进行管理的。

> 可以把它看作是一个 **智能的硬盘分区工具**，它允许你更加灵活地管理硬盘空间。LVM 允许你将多个硬盘或硬盘分区合并成一个大的虚拟硬盘，还可以随时调整硬盘大小，比如扩展或缩小硬盘，不需要重新格式化

LVM的基本组成部分

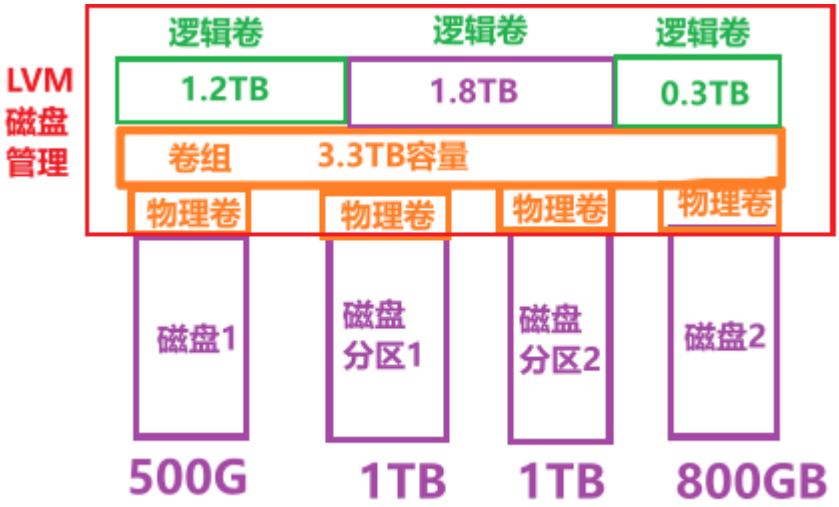
```
物理卷（PV）：将磁盘纳入给LVM进行管理
    物理卷是 LVM 管理的基础，它是硬盘、分区或 RAID 的基本单位。通过 pvcreate 命令创建物理卷。
    例如，如果你有一个硬盘 /dev/sdb，你可以将它变成一个物理卷（PV）
卷组（VG）：
    卷组是由一个或多个物理卷组成的逻辑单元。它将多个物理卷的空间合并为一个池，提供给逻辑卷使用。通过 vgcreate 命令创建卷组。

逻辑卷（LV）：
    逻辑卷是用户创建和使用的卷，相当于传统分区的功能。它通过卷组的空间来创建，支持动态扩展或缩小。通过 lvcreate 命令创建逻辑卷。
```

LVM 的优势

- **灵活性**：可以动态扩展和缩小逻辑卷。
- **简化管理**：LVM 将多个物理卷整合到一个卷组中，简化了磁盘管理。

- **快照功能**：可以为逻辑卷创建快照，方便备份。
- **分区整合**：可以将多个物理卷合并为一个卷组，提供统一的磁盘管理。




## 6.2 LVM的基本使用

假设有三块硬盘，想将它们结合起来并创建一个大存储空间来存储文件。通过 LVM，你可以把这三块硬盘当作一个大硬盘来使用。

- 1- 在VMware中，针对node3 额外构建三台磁盘， 每块磁盘的大小为20GB 20GB 30GB



```
[root@node3 ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                  11:0    1 10.8G  0 rom
nvme0n1             259:0    0  100G  0 disk
├─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
  ├─cs_bogon-root   253:0    0   95G  0 lvm  /
  └─cs_bogon-swap   253:1    0    4G  0 lvm  [SWAP]
nvme0n2             259:3    0   20G  0 disk
└─nvme0n2p1         259:4    0    4G  0 part [SWAP]
nvme0n3             259:5    0   20G  0 disk
nvme0n4             259:6    0   20G  0 disk
nvme0n5             259:7    0   30G  0 disk
```

- 2- 创建物理卷（PV）：首先，需要把每块硬盘变成物理卷。

```
pvcreate /dev/nvme0n3 /dev/nvme0n4 /dev/nvme0n5


注意说明：
    这里是直接针对整个磁盘设置为物理卷， 当然，如果你希望针对某个磁盘中的某个分区也是被支持的.
    通常的做法是直接使用整体磁盘
```

```
[root@node3 ~]# pvcreate /dev/nvme0n3 /dev/nvme0n4 /dev/nvme0n5
  Physical volume "/dev/nvme0n3" successfully created.
  Physical volume "/dev/nvme0n4" successfully created.
  Physical volume "/dev/nvme0n5" successfully created.
```

- 3- 创建卷组（VG）：然后将这些物理卷合并成一个卷组

```
vgcreate vg_data /dev/nvme0n3 /dev/nvme0n4 /dev/nvme0n5

vg_data:  卷组的名称，可以自定义
```

```
[root@node3 ~]# vgcreate vg_data /dev/nvme0n3 /dev/nvme0n4 /dev/nvme0n5
  Volume group "vg_data" successfully created
```

- 4- 创建逻辑卷（LV）：在卷组 `vg_data` 中创建一个逻辑卷。这就相当于在一个大的虚拟硬盘中创建了一个小的分区。

```
lvcreate -L 50G -n lv_data vg_data
```

```
2
3   50G 是逻辑卷的大小，lv_data 是逻辑卷的名字。
```

```
[root@node3 ~]# lvcreate -L 50G -n lv_data vg_data
  Logical volume "lv_data" created.
```

- 5- 格式化并挂载: 最后需要格式化逻辑卷，并挂载到系统中使用。

```
1   # 格式化操作:
2   mkfs.xfs /dev/vg_data/lv_data
3   # 创建挂载的目录位置
4   mkdir -p /mnt/data
5   # 执行挂载
6   mount /dev/vg_data/lv_data /mnt/data
```

```
[root@node3 ~]# mount /dev/vg_data/lv_data /mnt/data
[root@node3 ~]# df -h
文件系统                    容量  已用  可用 已用% 挂载点
devtmpfs                    4.0M     0  4.0M    0% /dev
tmpfs                       1.8G     0  1.8G    0% /dev/shm
tmpfs                       726M  9.1M  717M    2% /run
/dev/mapper/cs_bogon-root    95G  3.8G   92G    4% /
/dev/nvme0n1p1              960M  233M  728M   25% /boot
tmpfs                       363M  4.0K  363M    1% /run/user/0
/dev/mapper/vg_data-lv_data  50G  389M   50G    1% /mnt/data
```

```
1   LVM 就是通过将硬盘“虚拟化”，让你更灵活地管理磁盘空间。它不像传统的硬盘分区那样死板，你可以轻松地调整磁盘空间，甚至可以将多个硬盘合并成一个大的存储池来使用。通过简单的命令，你可以扩展、缩小或管理存储空间。
```

## 6.3 LVM的其他操作

- 1- 扩展卷组：

  在构建物理卷（PV）后，还有剩余空间未划分为卷组，你可以随时向现有的卷组中添加新的物理卷，扩展卷组的空间。

```
1   步骤一： 确保新的磁盘或分区已经被创建并初始化为物理卷（PV）
2   pvcreate 设备名称
3
4   步骤二： 使用 vgextend 命令将新的物理卷添加到现有的卷组中
```

```
vgextend 现有卷组名称 设备名称

步骤三：即可查看卷组空间
vgdisplay 卷组空间
```

```
[root@node3 ~]# vgdisplay vg_data
  --- Volume group ---
  VG Name               vg_data   卷组的名称
  System ID
  Format                lvm2    卷组的格式
  Metadata Areas        3     卷组的元数据区域数
  Metadata Sequence No  2     组元数据的版本号 每次卷组配置更改时，这个值会递增
  VG Access             read/write 表示当前卷组的访问权限
  VG Status             resizable 卷组的状态。resizable 表示卷组的大小可以动态扩展。
  MAX LV                0  卷组允许的最大逻辑卷数 0 表示没有设置最大限制
  Cur LV                1   当前在该卷组中已有的逻辑卷数量
  Open LV               1  已经打开的逻辑卷数量，通常在系统启动时，会自动打开卷组中的所有逻辑卷。
  Max PV                0 卷组最大可以包含的物理卷数量。0 表示没有设置最大数量
  Cur PV                3  当前卷组中包含的物理卷数量
  Act PV                3  当前处于活动状态的物理卷数量
  VG Size               <69.99 GiB 表示卷组的总大小
  PE Size               4.00 MiB 每个物理扩展（PE）的大小。PE 是 LVM 存储的基本单位
  Total PE              17917  卷组总共有多少个 PE，总共为 17917 个。
  Alloc PE / Size       12800 / 50.00 GiB  使用
  Free  PE / Size       5117 / <19.99 GiB  剩余
  VG UUID               2aLjeP-976g-vBNu-OiSS-3n7A-gktC-ka5v4z
```

- 2- 扩展逻辑卷（LV）

  可以增加某个逻辑卷（LV）的大小，以便使用更多的存储空间。

```
lvextend -L +10G /dev/vg_data/lv_data

# 使用 lvextend 命令来扩展逻辑卷。假设你要将逻辑卷 lv_data 扩展 10GB
```

```
[root@node3 ~]# lvextend -L +10G /dev/vg_data/lv_data
  Size of logical volume vg_data/lv_data changed from 50.00 GiB (12800 extents) to 60.00 GiB (15360 extents).
  Logical volume vg_data/lv_data successfully resized.
```

```
[root@node3 ~]# df -h
文件系统                  容量  已用  可用 已用% 挂载点
devtmpfs                  4.0M     0  4.0M    0% /dev
tmpfs                     1.8G     0  1.8G    0% /dev/shm
tmpfs                     726M  9.1M  717M    2% /run
/dev/mapper/cs_bogon-root  95G  3.8G   92G    4% /
/dev/nvme0n1p1            960M  233M  728M   25% /boot
tmpfs                     363M  4.0K  363M    1% /run/user/0
/dev/mapper/vg_data-lv_data  50G  389M   50G    1% /mnt/data  并未扩展
```

```
扩展后，需要重新调整文件系统的大小，才能真正使用新增的空间。
如果是 xfs 文件系统，使用 xfs_growfs  命令：
xfs_growfs /dev/vg_data/lv_data

如果是 ext4 文件系统，使用 resize2fs 命令：
resize2fs /dev/vg_data/lv_data
```

```
[root@node3 ~]# xfs_growfs /dev/vg_data/lv_data
meta-data=/dev/mapper/vg_data-lv_data isize=512    agcount=4, agsize=3276800 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=13107200, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
data blocks changed from 13107200 to 15728640
[root@node3 ~]#
[root@node3 ~]# df -h
文件系统                    容量  已用  可用 已用% 挂载点
devtmpfs                    4.0M     0  4.0M    0% /dev
tmpfs                       1.8G     0  1.8G    0% /dev/shm
tmpfs                       726M  9.1M  717M    2% /run
/dev/mapper/cs_bogon-root    95G  3.8G   92G    4% /
/dev/nvme0n1p1              960M  233M  728M   25% /boot
tmpfs                       363M  4.0K  363M    1% /run/user/0
/dev/mapper/vg_data-lv_data  60G  461M   60G    1% /mnt/data
```

- 3- 移除物理卷

  如果你不再需要某个物理卷，可以将其从卷组中移除。

```
1- 在移除物理卷之前，必须确保该物理卷上的数据已被迁移到其他物理卷。可以使用 pvmove 命令将数据迁移到其他物理卷：
例如： pvmove /dev/nvme0n3
# 注意： 该操作会将/dev/nvme0n3物理卷的数据迁移到该物理卷所属的卷组的其他物理卷中

2- 然后，使用 vgreduce 命令从卷组中移除物理卷：
vgreduce vg_data /dev/nvme0n3

3- 最后，使用 pvremove 命令删除物理卷：
pvremove /dev/nvme0n3
```

- 4- 快照

  LVM 还支持创建逻辑卷的快照（Snapshot），可以用来备份当前状态或测试环境。快照会保存当前逻辑卷的只读副本。

```
创建快照：
lvcreate -L 1G -s -n lv_data_snapshot /dev/vg_data/lv_data

-L 1G：指定快照的大小为 1GB。
-s：表示创建快照。
-n lv_snapshot：指定快照的名称。
/dev/vg_data/lv_data：指定要创建快照的原始逻辑卷。

这个命令将创建一个名为 lv_data_snapshot 的快照，大小为 1GB。你可以对这个快照进行操作，确保原始数据不受影响。

查看快照： 创建快照后，可以使用 lvdisplay 查看逻辑卷和快照的状态：
lvdisplay /dev/vg_data/lv_snapshot

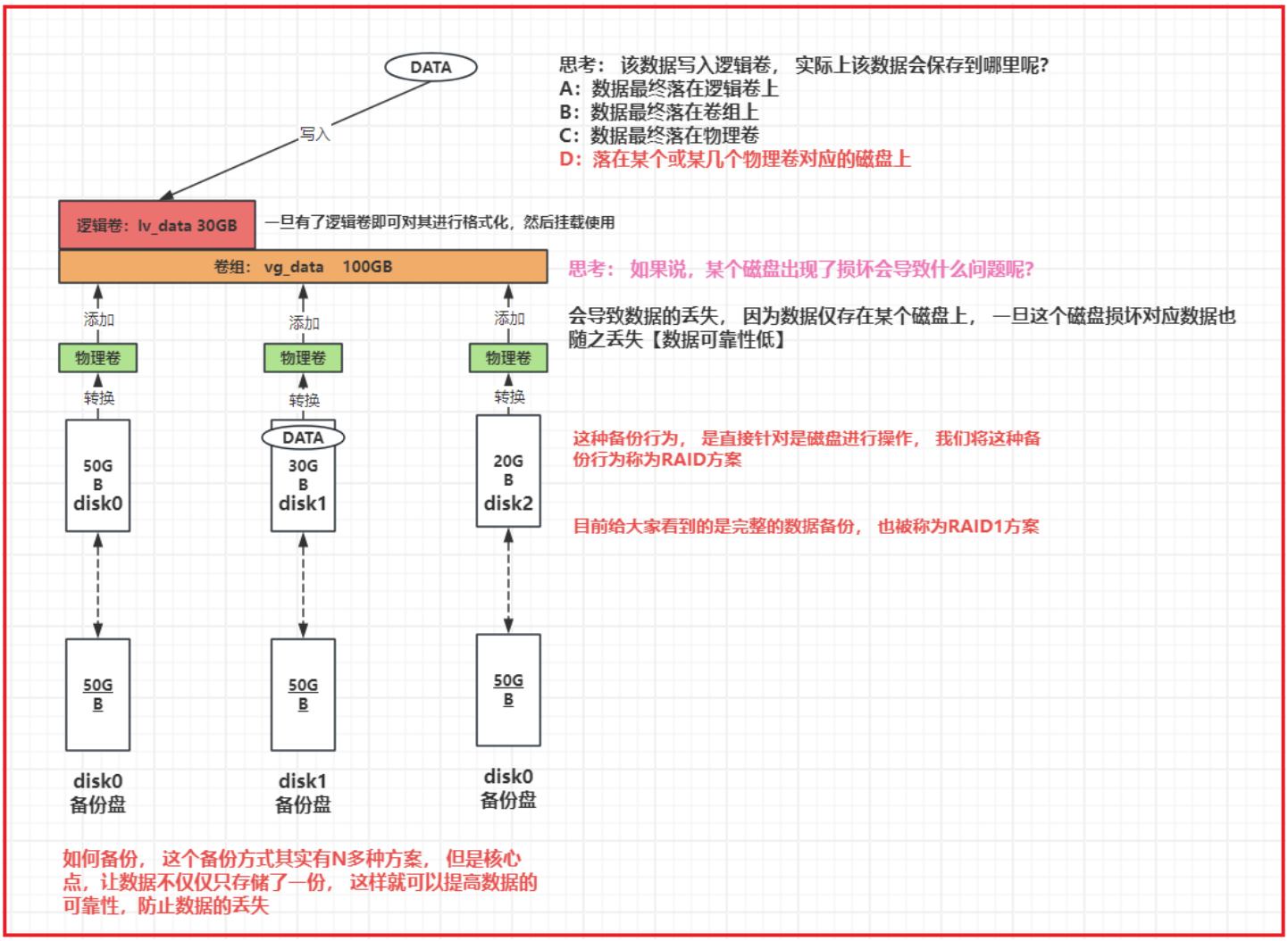
删除快照： 使用快照完成操作后，可以通过 lvremove 命令删除快照：
lvremove /dev/vg_data/lv_snapshot

扩展快照： 如果快照大小不足以保存所有增量数据，你可以扩展快照的大小：
lvextend -L +1G /dev/vg_data/lv_snapshot

恢复到快照： 如果你需要恢复到快照的状态，可以将原始逻辑卷恢复到快照的数据。假设你要恢复到 /dev/vg_data/lv_snapshot 快照：
lvconvert --merge /dev/vg_data/lv_snapshot
```

## 7. RAID相关内容

RAID 是一种将多个物理磁盘合并为一个逻辑磁盘的方法。它通过不同的配置来提高磁盘的性能、容错性、冗余等。RAID 的主要目标是提升系统性能和数据的可靠性。

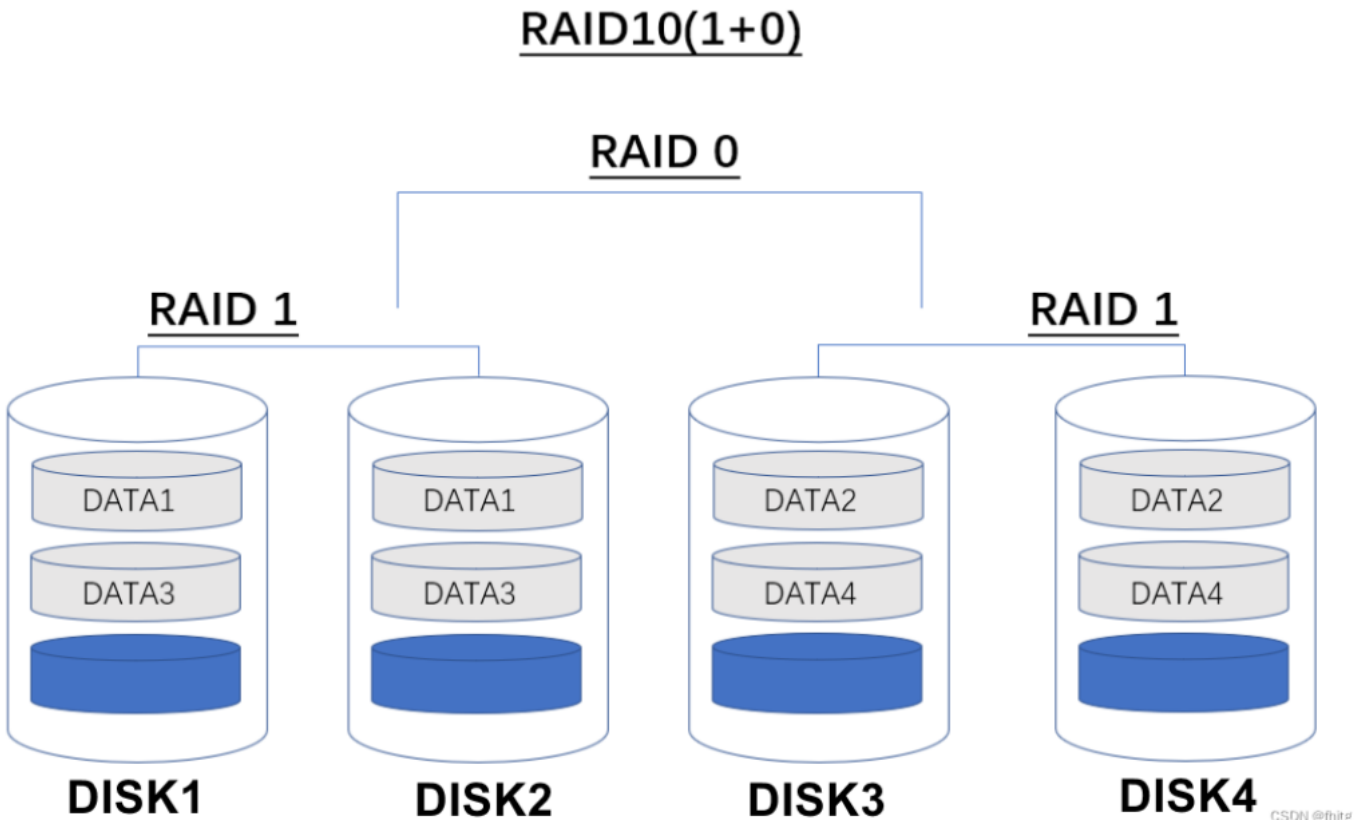



不同的RAID处理方式， 我们称为RAID的级别， 而级别主要由： RAID0、RAID1、RAID5、RAID10.....

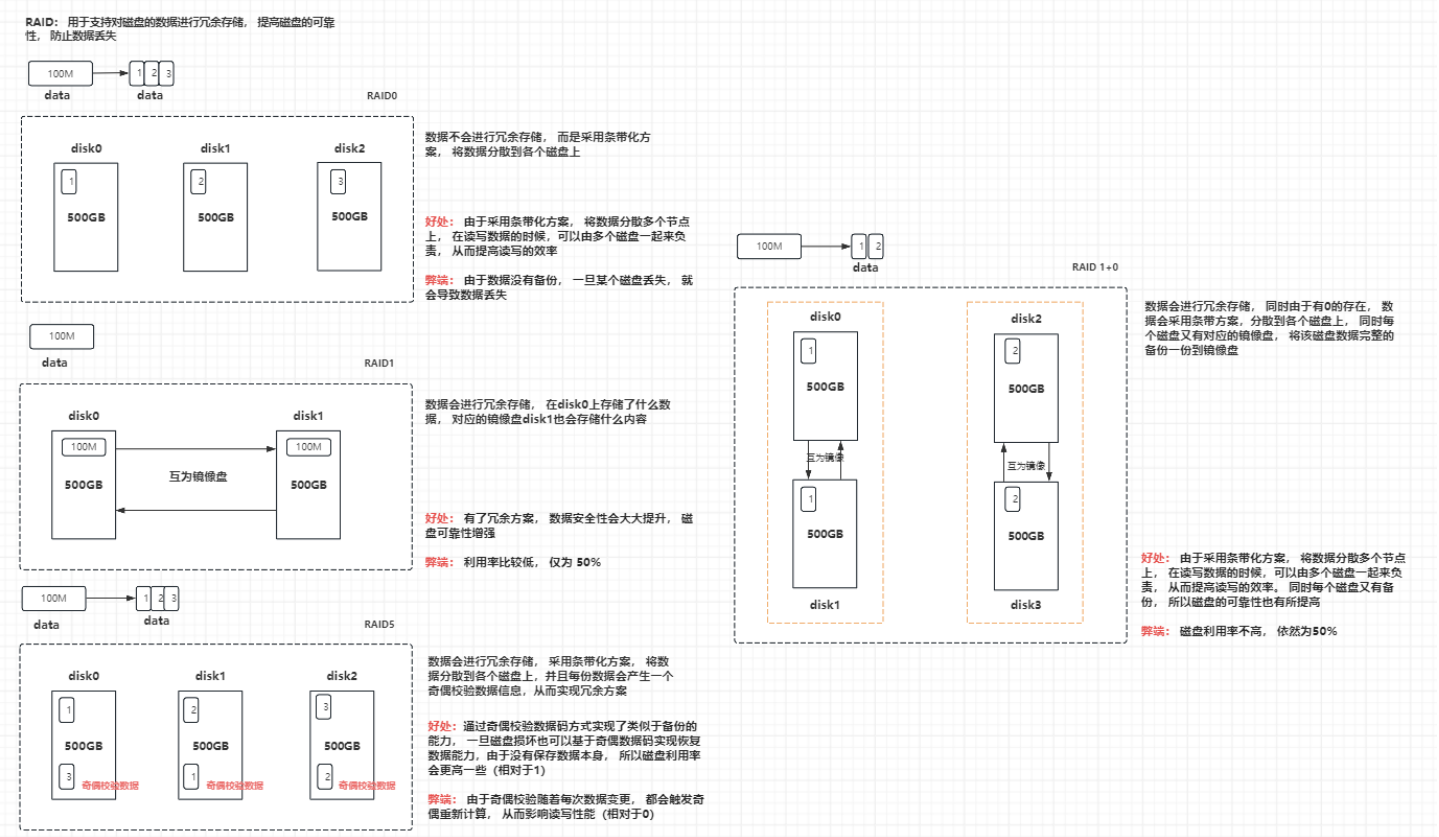
## 7.1 RAID的级别

RAID 有多个不同的级别，每个级别有其独特的优势和应用场景。常见的 RAID 级别包括：

- RAID 0（条带化）：将数据分散到多个硬盘上，提高读写性能，但没有冗余，任一硬盘故障会导致数据丢失。
- RAID 1（镜像）：将数据完整地复制到两个硬盘上，提供冗余性，保证数据安全，但性能提升有限。
- RAID 5（带奇偶校验的条带化）：通过在硬盘中存储奇偶校验数据，提供容错能力。通常至少需要三个硬盘。
- RAID 10（1+0，镜像+条带化）：结合了 RAID 1 和 RAID 0 的优点，提供性能和冗余性，但需要至少四个硬盘。




| RAID 级别 | **性能** | **冗余性** | **存储空间利用率** | **适用场景** | **优缺点** |
| --- | --- | --- | --- | --- | --- |
| **RAID 0** | 提升读写性能，数据分布到多个磁盘并行处理 | 无冗余，单个磁盘故障导致数据丢失 | 100% | 临时数据存储、高性能应用 | **优点**：性能高，存储空间利用率高；**缺点**：无冗余，硬盘故障会导致数据丢失 |
| **RAID 1** | 读取性能提升，但写入性能无显著变化 | 数据完全镜像，单个磁盘故障数据不丢失 | 50% | 需要高冗余的应用，如数据库服务器 | **优点**：高冗余性，数据安全；**缺点**：存储空间利用率低 |
| **RAID 5** | 性能良好，读取性能高，写入性能较低（因奇偶校验） | 提供冗余，通过奇偶校验信息恢复数据，单个磁盘故障不会丢失数据 | （N-1）/N | 需要平衡性能、冗余和存储空间的应用，如文件服务器 | **优点**：冗余性好，性能较高，存储空间利用较好；**缺点**：写入性能稍低，奇偶校验计算开销 |
| **RAID 10** | 性能提升，结合了 RAID 1 的冗余和 RAID 0 的性能 | 提供冗余，至少两个磁盘故障（不同组）不会丢失数据 | 50% | 高性能、高冗余要求的场景，如数据库和高可用服务器 | **优点**：高性能，高冗余性，故障容忍度高；**缺点**：存储空间利用率低，至少需要 4 个磁盘 |




## 7.2 软RAID和硬RAID

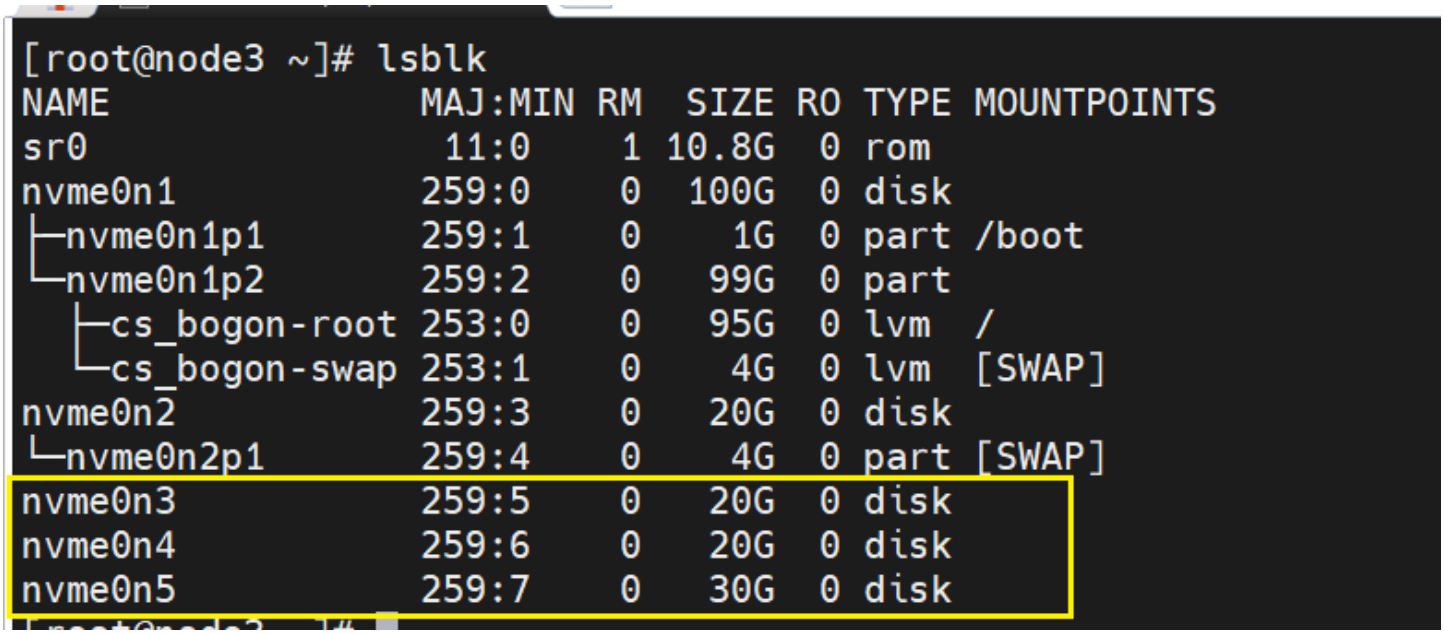
**硬件 RAID**：由专用的 RAID 控制卡处理，性能和管理通常较好，但成本较高。硬件 RAID 会在硬件层面上处理数据的冗余和校验。

**软件 RAID**：由操作系统控制，无需额外硬件。通常性能稍差，但成本低，配置和管理较为灵活，适合大多数应用场景。

## 7.3 RAID实战【记录为主】

- 1- 查看当前磁盘情况：

```
lsblk
```




- 2- 安装用于RAID的工具

```
可以使用 mdadm 工具创建软件 RAID，首先安装 mdadm
dnf -y install mdadm
```

- 3- 创建RAID 1 示例： 如果想要创建 RAID 1，可以使用如下命令

```
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/nvme0n3 /dev/nvme0n4

选项：
    mdadm 是用于管理 Linux 软件 RAID 阵列的命令行工具。它用于创建、管理和监视 RAID 阵列。
    --create 参数告诉 mdadm 创建一个新的 RAID 阵列。
    /dev/md0 是创建的 RAID 阵列的设备文件名。在 Linux 中，RAID 阵列被识别为 /dev/mdX（其中 X 是 RAID 阵列的编号）。在这个例子中，md0 表示第一个 RAID 阵列。
    --level=1 指定 RAID 的级别为 RAID 1，即镜像。RAID 1 会将数据同时写入到两个硬盘上，实现数据冗余（即两个磁盘上的数据完全相同）。如果一个磁盘发生故障，另一个磁盘的数据依然可用。
    --raid-devices=2 表示 RAID 阵列中包含两个磁盘。RAID 1 需要至少两个磁盘，因此这里指定了两个磁盘 /dev/sdb 和 /dev/sdc。
    /dev/nvme0n3 /dev/nvme0n4 这两个是要加入 RAI1 阵列的物理磁盘设备文件。在这个例子中，/dev/sdb 和 /dev/sdc 是两个物理硬盘，你希望它们一起组成 RAID 1 阵列。
```

```
[root@node3 ~]# mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/nvme0n3 /dev/nvme0n4
mdadm: Note: this array has metadata at the start and
    may not be suitable as a boot device.  If you plan to
    store '/boot' on this device please ensure that
    your boot-loader understands md/v1.x metadata, or use
    --metadata=0.90
Continue creating array [y/N]? y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

- 4- 配置 RAID 启动： 为了确保 RAID 配置在系统重启后仍然有效，需要将 RAID 配置写入配置文件：

```
mdadm --detail --scan --verbose >> /etc/mdadm.conf
dracut --force


dracut --force:  一个用于重新生成 initramfs（初始内存文件系统）的命令。initramfs 是一个临时的根文件系统，用于在 Linux 内核启动时加载必要的驱动程序和工具，以便挂载真正的根文件系统。
```

---

扩展， 如果需要清除阵列

1. **停止 RAID 阵列**

首先，你需要停止 RAID 阵列。在删除阵列之前，确保其已经停止运行。

```
mdadm --stop /dev/md0
```

这会停止名为 `/dev/md0` 的 RAID 阵列。如果使用的是其他设备名称（例如 `/dev/md1`），请相应地更改命令中的设备名称。

2. **删除 RAID 阵列**

接下来，可以删除 RAID 阵列。使用以下命令来删除它：

```
mdadm --remove /dev/md0
```

这会从系统中删除 `/dev/md0` 阵列。如果使用的是其他设备名称，请更改命令中的设备名称。

3. **清除 RAID 配置文件**

如果你曾将 RAID 配置信息写入了 `/etc/mdadm.conf` 文件，可以手动清除其中相关的条目。如果文件存在，删除包含 RAID 阵列信息的行或直接删除该文件。

```
rm /etc/mdadm.conf
```

4. **清除磁盘上的 RAID 元数据**

RAID 阵列中的磁盘通常会保存元数据来标识它们属于 RAID 阵列。如果你打算重新使用这些磁盘或将其从 RAID 阵列中删除，可以清除这些元数据。你可以使用 **`mdadm`** 的 `--zero-superblock` 命令来清除元数据。

例如，清除 `/dev/nvme0n3` 和 `/dev/nvme0n4` 上的 RAID 元数据：

```
mdadm --zero-superblock /dev/nvme0n3
mdadm --zero-superblock /dev/nvme0n4
```

这将清除磁盘上的 RAID 配置信息，确保它们不再被认为是 RAID 阵列的一部分。

5. **更新 initramfs**

如果你修改了 RAID 配置，尤其是清除了阵列，建议更新 initramfs 以确保系统启动时没有 RAID 配置遗留的问题。可以使用 `dracut` 命令来更新 initramfs：

```
dracut --force
```

6. **检查阵列状态**

使用以下命令确认 RAID 阵列已成功删除：

```
mdadm --detail --scan
```

此命令不应显示已删除的 RAID 阵列。